

DPI-LS

Engine + Embeddable Appraisal Widget

A pure scoring engine, a canonical observation contract,
and a drop-in web component for any AI agent

The DPI-LS (Digital FTE Performance Index, Life Sciences) is a real-time trust score for AI agents. It produces a 0–100 rating across seven dimensions — Productivity, Quality, Execution, Governance, Risk, Validation, and Cost — with hard compliance floors on G, R, and V, plus a coverage cap that prevents a high score from a single observed dimension. The composite is a weighted geometric mean with weight redistribution when a metric is `None`.

The system ships as a Python service (FastAPI + SQLAlchemy + Pydantic v2) and a framework-free vanilla web component. The whole architecture rests on one hard rule: the engine *never* imports an agent framework or a source SDK. Adapters translate any source into a canonical `AgentObservation` (full) or `PartialObservation` (one dimension’s worth); a merger combines latest-per-dimension partials before the engine sees them. Onboarding a new agent is writing a YAML field-mapping or a small `SourceAdapter` — never an engine change.

Status: working demo. 121 tests (<2s). Single SQLite-backed API serves a live board, per-agent cards, a conversational SME quality prompt, and a settings form.

Contents

1	Problem statement	4
1.1	What the customer actually has	4
1.2	What the customer cannot answer today	4
1.3	What we built	4
1.4	Why a plugin and not a platform	4
2	Solution overview	4
2.1	The score in one paragraph	4
2.2	Compliance gates (the part that matters for Life Sciences)	5
2.3	Bands	5
2.4	Worked examples (the four spec reference cases)	5
3	Architectural north star: agent-agnostic	5
4	High-level architecture	6
4.1	System diagram	6
4.2	The single most important design choice	6
5	Repository layout (as built)	7
6	The canonical contract	7
6.1	Full observation	7
6.2	Partial observation (the dimension-per-source pattern)	8
6.3	The Rating schema (engine output)	8
6.4	Settings (the only tunables)	8

7	The engine	9
7.1	Module map	9
7.2	The composite (the one piece worth quoting)	9
7.3	Flow	9
8	The ingestion layer: three adapter families	9
8.1	Family A — Full adapters	11
8.2	Family B — Source adapters (the dimension-per-source pattern)	11
8.3	Family C — Conversational SME quality	11
8.4	Generic webhook + YAML mapping (the zero-code path)	11
9	The dimension toolchain: open-source tools for each of the 7 scores	12
9.1	E — Execution: Langfuse v3 + OpenLLMetry	12
9.2	Q — Quality: DeepEval + MLflow (with SME conversational fallback)	13
9.3	G — Governance: Puvi Noise, Arize, NeMo Guardrails	14
9.4	R — Risk: ServiceNow + Jira (and BMC Remedy, PagerDuty)	14
9.5	V — Validation: DeepEval + MLflow (and Great Expectations)	15
9.6	C — Cost: AWS Cost Explorer + OpenCost (and Infracost, Cloudability)	15
9.7	P — Productivity: human baseline + output counter (no third-party required)	16
9.8	The dimension toolchain at a glance	17
9.9	The minimum viable toolchain	17
10	API surface (the wire)	17
11	The widget	18
11.1	Components shipped	18
12	End-to-end data flow (observation-based)	19
13	Storage schema	19
14	Plugin integration playbook	19
14.1	Path 0 — off-the-shelf source connector (zero code, ongoing)	19
14.2	Path 1 — direct canonical POST (zero code, one-time)	20
14.3	Path 2 — OTel adapter (zero code, drop-in for OTel emitters)	20
14.4	Path 3 — generic webhook + YAML mapping (zero Python, half a day)	20
14.5	Path 4 — SDK in the agent process (small code, best latency)	20
14.6	Path 5 — full adapter (only when the framework deserves it)	21
14.7	Path 6 — source adapter for a single dimension (one engineer, one day)	21
14.8	Decision matrix	22
15	How to get all 7 scores — the bootstrap playbook	22
15.1	What the Rating looks like during the bootstrap	22
15.2	The four-week bootstrap (recommended cadence)	23
15.3	The two failure modes to design around	25
15.4	What “all 7 scores” means to the customer	25
16	Gaps, loopholes, and the work plan	25
16.1	Gaps in the shipped demo	25
16.2	Loopholes and edge cases	26
16.3	Prioritised work plan	27
16.4	The two non-negotiables	27

17 Next steps: from working demo to production	27
17.1 Status today — what is already done	27
17.2 Gaps that block production deployment	28
17.3 What we can add to extend the architecture	29
17.4 Prioritised next-task list	29
17.5 The two-week sprint that unblocks the most	30
17.6 Sequencing principles	30
17.7 What the architecture explicitly does not need	31
18 Test surface (the proof)	31
19 Configuration	31
20 How to run the demo (90 seconds)	32
21 Conclusion	32
21.1 The single architectural assertion	32
21.2 What this document proves	32
21.3 What this document does not prove	33
21.4 The path from this document to a paying customer	33
21.5 The thing to remember on a bad day	33
A Appendix: full canonical AgentObservation (Pydantic v2)	34
B Appendix: dimension scoring formulas	35

1 Problem statement

1.1 What the customer actually has

A regulated enterprise — Regeneron is the anchor customer; the pattern repeats across Life Sciences, Financial Services, and Healthcare — has begun running AI agents in production. These agents do real work: file tickets, draft regulatory submissions, query internal systems, escalate incidents. They are not toys. They are digital employees.

1.2 What the customer cannot answer today

Four basic questions, none of which the market answers:

1. **Is it good?** Is the work accurate, consistent, and not made up?
2. **Is it safe?** Does it follow our policies? Does it cause incidents? Is it audit-ready?
3. **Is it worth it?** Is the cost lower than the human it replaced?
4. **Should we trust it more or less than last week?**

1.3 What we built

A **real-time report card for AI agents**, plus a widget that drops into any host page. Hand it an agent (or a stream of agent observations); it returns a 0–100 score, a per-dimension breakdown, and a compliance status. The demo sentence that defines done:

Hand it an agent — I'll score it live.

1.4 Why a plugin and not a platform

The customer is not going to rip out n8n, LangGraph, or their incident-management tool. We win by *adhering* to the customer's existing stack. The widget is the wedge; the integration story is “emit an observation” — and an *observation* can be a single JSON POST, an OpenTelemetry span, a complete canonical record, or a partial slice from one of N independent systems.

2 Solution overview

2.1 The score in one paragraph

The DPI-LS is a weighted geometric mean of seven sub-metrics, each normalised to [0,1]:

$$DPI-LS = 100 \times P^{0.15} \cdot Q^{0.20} \cdot E^{0.15} \cdot G^{0.20} \cdot R^{0.15} \cdot V^{0.10} \cdot C^{0.05} \quad (1)$$

The weights sum to 1.0. Quality and Governance carry the most weight; Cost the least. A weighted geometric mean (not arithmetic) means a single weak dimension *drags* the whole score down — you cannot buy your way out of a compliance failure with good economics.

Two refinements the engine makes on the textbook formula:

- **Weight redistribution on missing metrics.** When a metric is `None` (e.g. `Q` not yet captured), its weight is dropped and the remaining weights are renormalised to sum to 1.0. The engine never treats missing data as zero.
- **Coverage cap.** If fewer than `min_dimensions_for_full_band` (default 5) dimensions are present, the band is capped at “Needs Optimization” with a human-readable reason on the **Rating**. This prevents a single high-quality observation from scoring “Exceptional.”

2.2 Compliance gates (the part that matters for Life Sciences)

Three dimensions have hard floors:

$$\text{Unsafe} \iff (G < 0.60) \vee (R < 0.50) \vee (V < 0.60) \quad (2)$$

If any floor is breached, the score is capped at 69 and the agent is flagged *Unsafe* regardless of the raw geometric mean. Tunable per-deployment via `Settings.gate_thresholds`, but the engine respects what the customer configured — no client-side override.

2.3 Bands

Band	Range
Exceptional	85–100
Strong	70–84
Needs Optimization	50–69
Underperforming / Unsafe	below 50

2.4 Worked examples (the four spec reference cases)

These four outputs are asserted in `tests/test_engine_reference.py`. If any break, the engine is wrong.

Profile	P	Q	E	G	R	V	C	DPI	Status
Strong	0.85	0.85	0.85	0.85	0.85	0.85	0.85	85	Strong
Excellent	0.92	0.92	0.92	0.92	0.92	0.92	0.92	92	Exceptional
Mediocre	0.55	0.55	0.55	0.55	0.55	0.55	0.55	55	Needs Opt.
Governance failure	0.85	0.85	0.85	0.25	0.85	0.85	0.85	67	Unsafe

The last row is the critical one. Without the gate, the raw geometric mean would be 80 (“Strong”). With the gate enforcing $G < 0.60 \Rightarrow$ cap at 69, the score drops to **67** and the agent is flagged *Unsafe*. This is the test case the engine must pass on day 1.

3 Architectural north star: agent-agnostic

The engine must **not** know or care how an agent is built. Three rules, internalised before writing any code:

1. **Canonical Telemetry Contract** — one normalised Pydantic schema. The engine computes the 7 metrics *only* from this object.
2. **Adapters** — thin plugins that translate one source (an agent runtime OR an observability/ITSM tool) into the canonical contract. Swappable, discovered via a registry.
3. **Universal fallback** — a `GenericWebhookAdapter` driven by a *declarative YAML field-mapping*, plus an OpenTelemetry ingest. Any agent that can POST JSON or emit OTel spans is supported with **zero custom code**. Framework-specific adapters are *optimisations layered on top*, not a prerequisite.

Hard rules:

- The engine core depends **only** on the contract. Never **import** an agent framework into `engine/`.

- Onboarding a new agent = a new adapter **or** a new YAML mapping. *Never* an engine change.
- The engine is pure and fully unit-tested. No network or DB calls in **engine/**. Tests assert the four reference outputs.

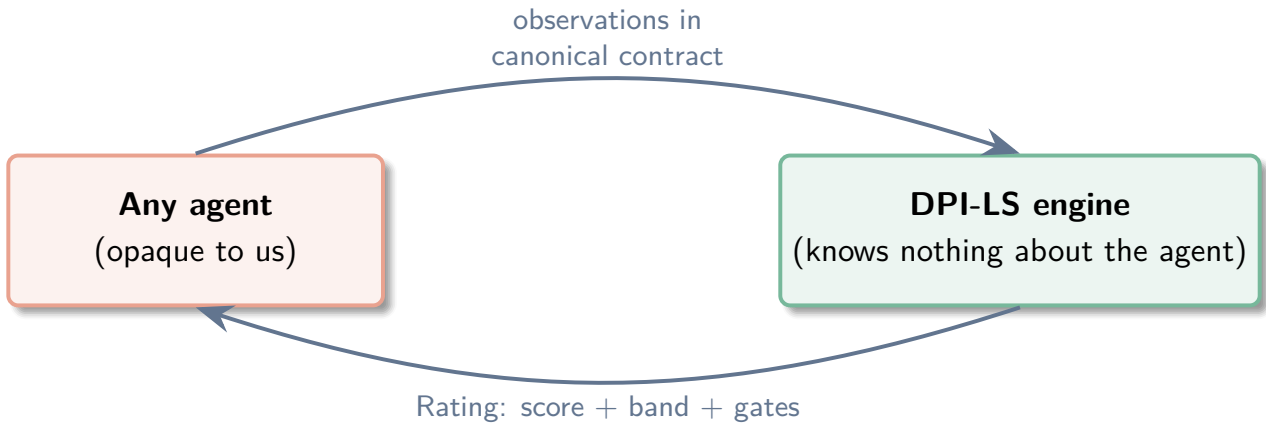


Figure 1: The agent platform is opaque. The DPI-LS engine consumes *only* `AgentObservation` / `PartialObservation` objects. Nothing about the agent leaks past the adapter.

4 High-level architecture

4.1 System diagram

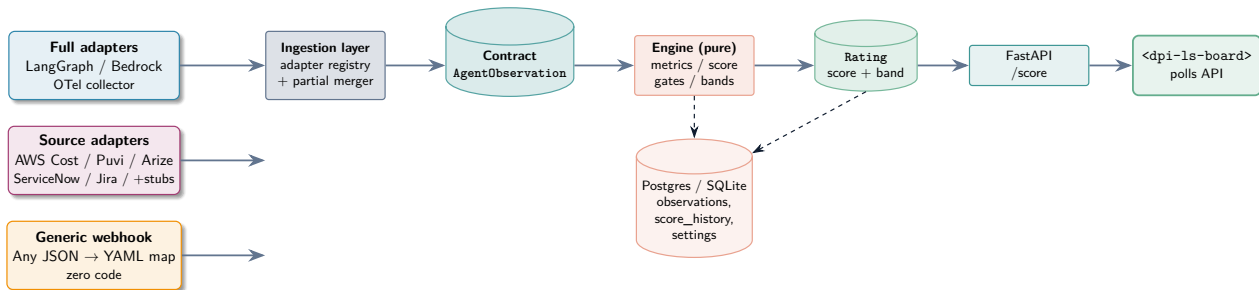


Figure 2: High-level architecture. Three adapter families converge on the ingestion layer. The contract is the only thing the engine sees. The `Rating` flows out to the API and the widget. Storage is the only side-effect.

4.2 The single most important design choice

The agent platform is not in the diagram above the dashed conceptual line. The ingestion layer is the *only* place that has to know the difference between, say, a ServiceNow JSON payload and an OTel span. Below the contract line, the rest of the system deals exclusively with canonical observations.

5 Repository layout (as built)

Directory	Purpose
contract/	Pydantic models — the canonical AgentObservation, PartialObservation, Rating, Settings.
engine/	PURE scoring. metrics.py, score.py, gates.py, bands.py, rate.py, sme_flow.py. No I/O.
ingestion/	Adapter base class + registry.
generic/	GenericWebhookAdapter (YAML mapping), OTelAdapter. Universal path.
sources/	aws_cost, puvi_noise, arize, servicenow, jira + 5 stubs (langgraph, bedrock, ray, bmc, sap_hr).
api/	FastAPI app, routes, orchestration, bootstrap.
store/	SQLAlchemy models + repo (agents, observations, partials, score_history, settings, sme_ratings, sme_flow_sessions).
widget/	dpi-ls.js (vanilla web components) + demo.html.
fixtures/	Synthetic payloads — clearly _label-tagged — used by tests and the demo.
tests/	121 tests, <2s, including the four DPI-LS reference outputs.
scripts/	demo_seed.sh — populates the board for the demo.

6 The canonical contract

Every adapter returns one of two things. A full AgentObservation when the source knows the whole snapshot, or a list of PartialObservation when a source only knows one dimension’s worth. The engine consumes nothing else. Null fields mean “needs other source”, never zero.

6.1 Full observation

Listing 1: contract/models.py — the canonical schema.

```

1 class TaskStats(BaseModel):
2     assigned: int; completed: int; failed: int
3     pending_approval: int = 0
4     blocked_no_access: int = 0
5
6 class Executions(BaseModel):
7     attempts: int; successful: int
8
9 class Policy(BaseModel):
10    total_actions: int
11    violations: list[PolicyViolation] = Field(default_factory=list)
12
13 class AgentObservation(BaseModel):
14    agent_id: str
15    agent_name: str
16    period_start: datetime
17    period_end: datetime
18    tasks: TaskStats
19    executions: Executions
20    policy: Policy
21    incidents: list[Incident] = Field(default_factory=list)
22    quality: Optional[Quality] = None # null = needs SME
23    validation: Validation
24    cost: Cost
25    source: str

```

6.2 Partial observation (the dimension-per-source pattern)

Most real sources know only one dimension. AWS Cost knows C. Puvi / Arize know G. ServiceNow / Jira know R. A `PartialObservation` carries as little as one dimension plus `agent_id`:

Listing 2: `contract/partial.py` — one source, one dimension.

```

1 class PartialObservation(BaseModel):
2     agent_id:      str
3     agent_name:    Optional[str] = None
4     period_start:  datetime
5     period_end:    datetime
6     source:        str
7     tasks:         Optional[TaskStats] = None
8     executions:    Optional[Executions] = None
9     policy:        Optional[Policy] = None
10    incidents:      Optional[list[Incident]] = None
11    quality:        Optional[Quality] = None
12    validation:     Optional[Validation] = None
13    cost:           Optional[Cost] = None

```

A merger combines the *latest per dimension* across all partials for an agent into a working `PartialObservation`; only then does the engine see it. The engine never has to reason about “missing C means C is zero.”

6.3 The Rating schema (engine output)

Listing 3: `contract/rating.py` — what comes out.

```

1 class Rating(BaseModel):
2     score:          float          # post-cap
3     raw_score:      float          # pre-cap composite
4     band:           str            # Exceptional / Strong / Needs
5                               # Optimization / Underperforming
6     unsafe:         bool           # any G/R/V floor breached
7     gate_failures:  list[str]
8     metrics:        dict[str, Optional[float]] # the 7 inputs
9     missing:        list[str]      # metric keys not yet observed
10    dimensions_measured: list[str]
11    coverage:        int            # 0..7
12    coverage_capped: bool
13    cap_reasons:     list[str]      # human-readable explanation per cap

```

6.4 Settings (the only tunables)

Listing 4: `contract/settings.py`.

```

1 DEFAULT_WEIGHTS:      dict = {"P":.15, "Q":.20, "E":.15, "G":.20, "R"
2     :.15, "V":.10, "C":.05}
3 DEFAULT_Q_SUB_WEIGHTS: dict = {"accuracy":.70, "consistency":.20, "
4     hallucination":.10}
5 DEFAULT_GATE_THRESHOLDS: dict = {"G":.60, "R":.50, "V":.60}
6
7 class Settings(BaseModel):
8     weights:          dict[str, float]
9     q_sub_weights:    dict[str, float]
10    gate_thresholds:   dict[str, float]
11    r_max:             float = 10.0

```



```

10     human_cost_per_output:          float = 1.0
11     utilization:                   float = 1.0
12     min_dimensions_for_full_band:  int   = 5      # coverage cap

```

All tunables live in the database `settings` table — one row — read and written through `GET /settings` and `PUT /settings`. *Nothing* is hardcoded in the engine.

7 The engine

The engine is a collection of **pure functions** in `engine/`. No imports from `api/`, `store/`, `ingestion/`. No network. No DB. Tests assert the four reference outputs and run in milliseconds.

7.1 Module map

Module	Responsibility
<code>metrics.py</code>	Normalise each of the 7 dimensions to [0,1].
<code>score.py</code>	Weighted geometric mean with weight redistribution.
<code>gates.py</code>	G/R/V floor check + Unsafe flag.
<code>bands.py</code>	Map score → band label.
<code>rate.py</code>	Per-metric sub-rate helpers used by <code>metrics</code> .
<code>sme_flow.py</code>	State machine for conversational quality capture.

7.2 The composite (the one piece worth quoting)

Listing 5: `engine/score.py` — weighted geometric mean.

```

1  def composite(metrics, weights=None):
2      weights = weights or DEFAULT_WEIGHTS
3      present = {k: v for k, v in metrics.items() if v is not None}
4      if not present:
5          return 0.0
6      total_weight = sum(weights[k] for k in present)
7      if total_weight <= 0:
8          return 0.0
9      product = 1.0
10     for k, v in present.items():
11         w = weights[k] / total_weight          # redistribute weight off
12         None                                     None
13         product *= max(float(v), _EPS) ** w    # tiny epsilon keeps 0 from
14         nuking                                   nuking
15     return 100.0 * product

```

Two non-obvious choices: (a) missing metrics have their weight redistributed proportionally — Q being absent does not collapse the score, and (b) a tiny epsilon (1×10^{-9}) replaces literal zero so one zero doesn’t drag the geometric mean to zero. The gates handle the “stop the world” case.

7.3 Flow

8 The ingestion layer: three adapter families

These three are not convenience adapters. **They are the integration story.** Any agent that can do one of these can be scored.

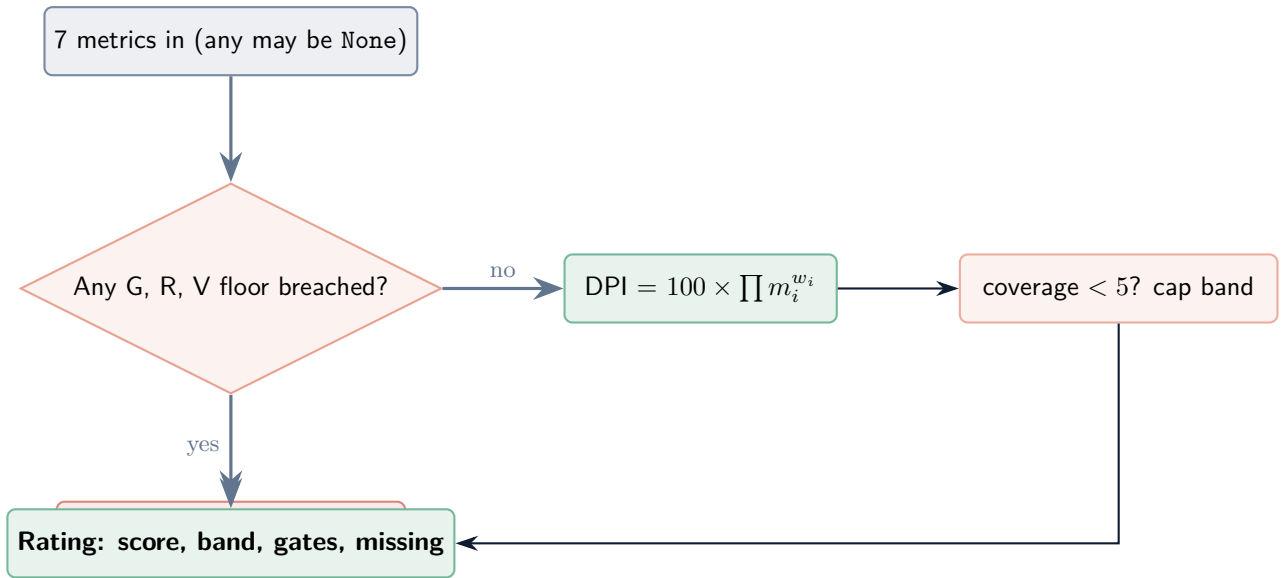


Figure 3: Engine flow. Gate check first, cap applies to the composite. A second pass adds the coverage cap before the **Rating** is returned.

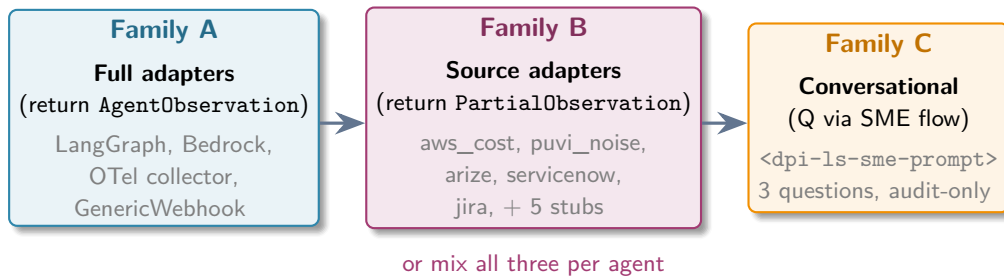


Figure 4: The three adapter families. An agent's score can be assembled from any combination.

8.1 Family A — Full adapters

A full adapter’s only job is to return one (or many) `AgentObservation`. Examples shipped:

- `OTelAdapter` — receives OTEL spans, decodes them via a shared field-mapping, produces canonical observations.
- `GenericWebhookAdapter` — bound to a YAML field-mapping, exposed as `webhook:<source>` for any JSON POST the customer wants to support.
- Framework-specific full adapters (LangGraph, Bedrock Agent Engines) plug in the same shape.

8.2 Family B — Source adapters (the dimension-per-source pattern)

The realistic case: no single system knows all seven dimensions. AWS Cost knows C. Puvi / Arize know G (and sometimes Q). ServiceNow / Jira know R. A `SourceAdapter` returns `list[PartialObservation]`; the API stores them, re-merges latest-per-dimension per agent, and re-scores. The engine never has to reason about “missing C means C is zero.”

Source adapters shipped:

Adapter	Provides
<code>aws_cost</code>	C (cloud spend / output)
<code>puvi_noise</code>	G (policy violations)
<code>arize</code>	G + Q (LLM observability)
<code>servicenow</code>	R (incidents)
<code>jira</code>	R (incidents)
<code>langgraph, bedrock, ray, bmc, sap_hr</code>	stubs (registered, return [])

Five real adapters are under 80 lines each.

8.3 Family C — Conversational SME quality

Quality (Q) is the dimension that cannot be reliably inferred from logs. A regulated customer needs an SME in the loop. The `<dpi-ls-sme-prompt>` component runs a 3-question conversational capture (accuracy → consistency → hallucination → review). Backed by `/sme-flow/start + /sme-flow/{id}/respond`, implemented as a state machine in `engine/sme_flow.py`. Audit-only: a direct SME rating via `POST /agents/{id}/sme-rating` also works.

8.4 Generic webhook + YAML mapping (the zero-code path)

If an agent runtime (or an aggregator) can produce JSON, no code is required. Write a YAML field-mapping and drop it into `MAPPINGS_DIR`. The bootstrap auto-registers it as `webhook:<source>` and exposes `/ingest/webhook:<source>`. Mapping syntax covers dot/bracket paths, `_each` for iteration, and `_optional` for nullable groups.

Listing 6: Excerpt of `fixtures/mapping_acme.yaml`.

```

1 name: acme
2 extract:
3   agent_id:      "$.meta.agent_id"
4   agent_name:    "$.meta.agent_name"
5   period_start:  "$.window.start"
6   period_end:    "$.window.end"
7   executions.attempts: "$.counters.attempts"
8   executions.successful: "$.counters.successful"
9   policy.total_actions: "$.counters.actions"
10  policy.violations._each: "$.violations[*]"

```

```

11 cost.ai_cost_per_output: "$.cost.per_output"
12 cost.cloud_cost:        "$.cost.total"
13 cost.systems_accessed:   "$.counters.systems"
14 _optional:
15     quality.accuracy:    "$.ratings.accuracy"
16     quality.consistency:  "$.ratings.consistency"
17     quality.hallucination_rate: "$.ratings.hallucination"

```

9 The dimension toolchain: open-source tools for each of the 7 scores

Each of the seven dimensions can be sourced from known best-of-breed open-source tools. Most agents do not need all seven on day 1; the bootstrap playbook (§??) orders them. This section lists the recommended tool per dimension, what it provides, and the DPI-LS integration path it slots into. *None of these are required — every one is swappable, and the engine does not know which one you picked.*

The pattern in every case is the same: the tool collects data in its own format; a thin `SourceAdapter` or a YAML field mapping (Path 3 or Path 6 of the integration playbook) translates it into `PartialObservation` entries; the DPI-LS merger combines them with whatever else the customer’s environment supplies.

9.1 E — Execution: Langfuse v3 + OpenLLMetry

What it scores: did the agent actually finish what it started? Inputs are `executions.attempts` and `executions.successful`.

- **Langfuse v3** (open source, Apache 2.0) — an LLM observability platform. Tracks every generation, tool call, and chain as a trace. Exposes a REST API and a Postgres backend. The v3 model treats “trace → span → generation” as first-class, which maps cleanly to `Executions(attempts, successful)`.
- **OpenLLMetry** (open source, Apache 2.0, by Traceloop) — OpenTelemetry instrumentation auto-injected for LLM frameworks (LangChain, LlamaIndex, OpenAI, Anthropic, Bedrock, etc.). Emits OTel spans with the OpenInference semantic conventions (`openinference.span.kind`, `llm.model_name`, etc.). The DPI-LS `OTelAdapter` consumes these directly — no per-framework adapter needed.

Integration: Path 2 (OTel) if the agent uses OpenLLMetry, or Path 5 (full adapter) with a 50-line wrapper around the Langfuse `/api/public/traces` endpoint. The repository’s `OTelAdapter` already handles the OpenInference attribute mapping for `gen_ai.task = true`.

Listing 7: Langfuse → E via a small puller. Skeleton.

```

1 # scripts/pull_langfuse.py --- runs on a schedule, posts partials.
2 # GET /api/public/traces?from_timestamp=...&limit=500
3 # for each trace:
4 #     attempts = len(trace.observations)
5 #     successful = sum(1 for o in observations if o.status == "SUCCESS")
6 # POST /ingest/source/langfuse
7 ---
8 source: langfuse
9 period_start: 2026-06-01T00:00:00Z
10 period_end:   2026-06-02T00:00:00Z
11 agents:
12     - agent_id: chandra
13       attempts: 137
14       successful: 129

```

9.2 Q — Quality: DeepEval + MLflow (with SME conversational fallback)

What it scores: is the work accurate, consistent, and not made up? Inputs are `quality.accuracy`, `quality.consistency`, `quality.hallucination_rate`.

- **DeepEval** (open source, Apache 2.0) — an LLM-evaluation framework. Ships metrics: GEval, AnswerRelevancy, Hallucination, Faithfulness, Toxicity, plus 20+ more. Returns 0–1 scores. Runnable as a pytest plugin.
- **MLflow** (open source, Apache 2.0) — the experiment tracking and model registry standard. `mlflow.evaluate()` runs LLM evaluations against a dataset, logging metrics to the MLflow Tracking Server. Plays well with the rest of the MLOps stack (feature store, model registry, registry webhooks).
- **LLM-as-judge** — both DeepEval and MLflow support “use one LLM to grade another”. Cost ~\$0.01 per sample. Tune prompt carefully; calibrate against SME ratings for the first month.
- **SME conversational flow** — the DPI-LS `<dpi-ls-sme-prompt>` component runs a 3-question capture (accuracy, consistency, hallucination, review) against a human in the loop. Slower but auditable; required for regulated industries regardless of LLM judge.

Integration: Path 6 (a 60-line `SourceAdapter` that runs DeepEval on a sample of recent agent outputs and posts one `PartialObservation` with the mean of each metric). For regulated industries, the SME flow runs alongside as the audit-of-record.

Listing 8: Skeleton: DeepEval → Q. `ingestion/sources/deepeval.py`.

```

1 from deepeval.metrics import (AnswerRelevancyMetric,
2                               HallucinationMetric)
3 from deepeval.test_case import LLMTestCase
4 from contract import PartialObservation, Quality
5 from .base import SourceAdapter
6
7 class DeepevalAdapter(SourceAdapter):
8     name = "deepeval"
9
10    def topartials(self, payload):
11        cases = payload["test_cases"] # [{"input":..., "actual":..., "
12                                     context":...}, ...]
13        rel = AnswerRelevancyMetric(); hal = HallucinationMetric()
14        acc_scores, hal_scores = [], []
15        for c in cases:
16            tc = LLMTestCase(input=c["input"],
17                             actual_output=c["actual"],
18                             retrieval_context=c.get("context"))
19            rel.measure(tc); hal.measure(tc)
20            acc_scores.append(rel.score); hal_scores.append(hal.score)
21        return [PartialObservation(
22            agent_id=payload["agent_id"],
23            period_start=payload["period_start"],
24            period_end=payload["period_end"],
25            source=self.name,
26            quality=Quality(
27                accuracy=sum(acc_scores)/len(acc_scores),
28                consistency=0.85, # from a paired-test
29                                set
30                hallucination_rate=sum(hal_scores)/len(hal_scores),
31            ),
32        )]
```

9.3 G — Governance: Puvi Noise, Arize, NeMo Guardrails

What it scores: does the agent follow the rules? Inputs are `policy.total_actions` and `policy.violations`.

- **Puvi Noise** (already wired in the repo) — the compliance team’s tool of record for policy violations.
- **Arize** (already wired) — LLM observability with `guardrail` span attributes and a policy-violation feed. Path 2 (OTel) covers it.
- **NeMo Guardrails** (NVIDIA, open source, Apache 2.0) — programmable guardrails in Colang. Emits OTel spans with `policy.violation` attributes that the DPI-LS OTel adapter maps directly to G.
- **Guardrails AI** (open source) — a lighter-weight validators library; useful for input/output PII detection, toxicity, etc. Emits structured logs.

Integration: Path 2 (OTel) for NeMo Guardrails and Arize (zero code). Path 5 (full adapter) for Guardrails AI’s JSON log format. Path 6 for Puvi.

9.4 R — Risk: ServiceNow + Jira (and BMC Remedy, PagerDuty)

What it scores: how often, and how badly, does the agent cause incidents? Inputs are `incidents` (a list of (`severity_weight`, `frequency`, `source`) tuples).

- **ServiceNow** (commercial) — ITSM. Pull incidents via the Table API filtered by `assignment_group=agents`. Path 6 (live boto3-equivalent fetch).
- **Jira** (commercial) — the other ITSM. Jira REST API with JQL filter. Path 6.
- **BMC Remedy** — the third legacy ITSM. Stub adapter already in repo (`ingestion/sources/bmc.py`), awaiting the REST fetch layer.
- **PagerDuty / Opsgenie** — on-call alerting. Map `incident.acknowledged` to R via a webhook subscription. Path 3 (YAML mapping on the PD webhook payload).

Integration: Path 6 in every case. Adapter counts incidents per agent over the period and weighs them by `severity_weight`:

Listing 9: Skeleton: Jira → R. `ingestion/sources/jira.py`.

```

1 from contract import Incident, PartialObservation
2 from .base import SourceAdapter
3
4 SEVERITY_WEIGHT = {"Low": 1, "Medium": 3, "High": 5, "Critical": 10}
5
6 class JiraAdapter(SourceAdapter):
7     name = "jira"
8
9     def to_partials(self, payload):
10         # payload is the Jira search result (already paged by the caller)
11
12         per_agent: dict[str, list[Incident]] = {}
13         for issue in payload.get("issues", []):
14             agent = issue["fields"]["customfield_agent_id"]
15             sev = issue["fields"]["priority"]["name"]
16             per_agent.setdefault(agent, []).append(Incident(
17                 severity_weight=SEVERITY_WEIGHT.get(sev, 1),

```

```

17         frequency=1, source="jira",
18     ))
19     return [PartialObservation(
20         agent_id=a, source=self.name,
21         period_start=payload["period_start"],
22         period_end=payload["period_end"],
23         incidents=incs,
24     ) for a, incs in per_agent.items()]

```

9.5 V — Validation: DeepEval + MLflow (and Great Expectations)

What it scores: is the agent audit-ready? Inputs are `validation.required_components`, `validation.validated_components`, `validation.audit_ready`.

- **MLflow** (open source) — the model registry. The `audit_ready` flag is the union of “model is registered, model is in Production stage, latest run is signed-off by an SME.” MLflow’s REST API exposes all of this.
- **DeepEval** — the same eval framework that feeds Q can also feed V: an eval suite that must pass before a model version is “validated” is the same suite that proves audit-readiness.
- **Great Expectations** (open source) — the data-quality standard. Useful for V when the agent’s outputs are structured data (CSV / DataFrame) and a data contract has been written.
- **Pandera** (open source) — a lighter pandas/Polars data-validation library. Use for tabular outputs.
- **TruEra** (commercial) — model-quality and explainability; exports “validated” flags.

Integration: Path 6 (a thin wrapper that queries MLflow’s `/api/2.0/mlflow/registered-models/...` for the agent’s model version, checks stage and latest run status, and posts one `PartialObservation` with the V fields populated).

9.6 C — Cost: AWS Cost Explorer + OpenCost (and Infracost, Cloudability)

What it scores: is it cheaper than a human for the same output? Inputs are `cost.ai_cost_per_output`, `cost.cloud_cost`, `cost.systems_accessed`.

- **AWS Cost Explorer** (commercial, AWS-native) — the source of truth for AWS spend. `GetCostAndUsage` with `GroupBy=TAG(Key=AGENT_ID)` returns per-agent spend. Already wired in the repo as `aws_cost` — the boto3 fetch step is the one piece still to add (P0 in the work plan).
- **OpenCost** (CNCF, open source) — the Kubernetes cost-monitoring standard. If the agent runs on k8s, OpenCost gives per-pod, per-namespace spend.
- **Infracost** (open source with paid tier) — Terraform cost estimation. Use at deploy time to predict spend; use the actual `infracost breakdown` JSON as a CI gate.
- **Cloudability** / **Kubecost** — multi-cloud cost allocators.

Integration: Path 6 in every case. The skeleton `aws_cost` adapter is in the repo (see §??); the only thing to add is the boto3 fetch loop.

9.7 P — Productivity: human baseline + output counter (no third-party required)

What it scores: did the agent produce as much as a human would? Inputs are a per-agent human-output baseline and the agent’s output count over the period.

- **Human baseline** — the customer supplies this once per agent via the `AgentBaseline` Pydantic model and the `/agents/{id}/baseline` endpoint (or directly in the `settings` table). It is an opinionated number (a senior clinician files 12 incidents per shift; a junior files 6). There is no third-party tool for it — it is organisational knowledge.
- **Output counter** — the source that already counts agent outputs is the same one feeding E (Langfuse, OpenLLMetry) or C (AWS → output count in the wire payload). Reuse it; do not add another tool.

Integration: No adapter needed for P itself. The output count comes from whichever source already feeds E; the baseline is a setting. $P = \min(1, \text{AI_output} / \text{human_baseline})$ with the baseline coming from the `AgentBaseline` table.

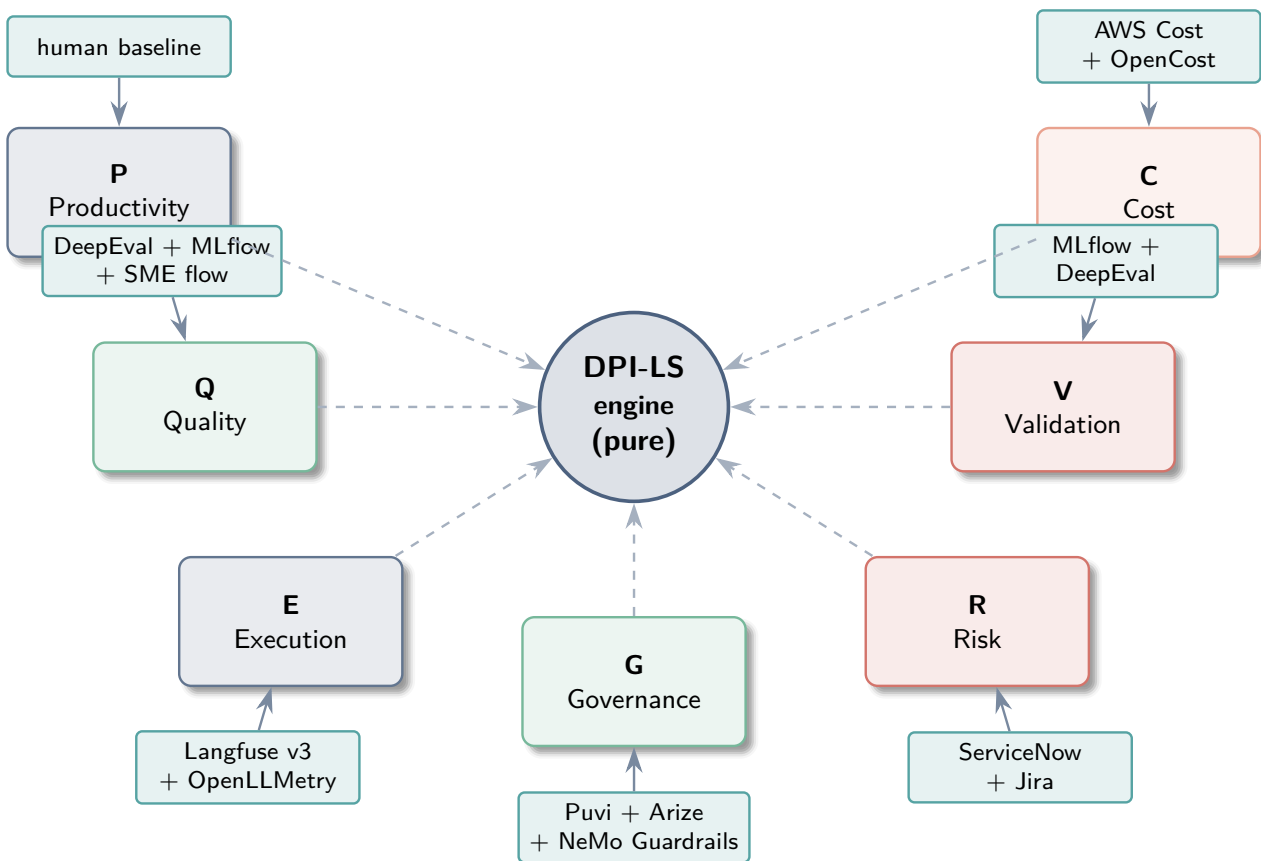


Figure 5: The dimension toolchain at a glance. Each of the seven dimensions is fed by a known open-source tool or by a customer configuration (P, the only dimension that does not need a third-party). The DPI-LS engine consumes only the canonical contract; it does not know which tool supplied which dimension. The three colour-coded dimensions (Q, G, R, V) are the ones with hard floors or the SME-flow input; the dashed arrows emphasise the contract boundary.

9.8 The dimension toolchain at a glance

Dim	Primary tool(s)	Path	Data shape	Status in repo
E	Langfuse v3 + OpenLLMetry	2 / 5 / 6	traces / spans	OTel adapter ready; Langfuse wrapper TBD.
Q	DeepEval + MLflow + SME flow	6	per-sample scores	SME flow ships; DeepEval adapter TBD.
G	Puvi Noise + Arize + NeMo Guardrails	2 / 5 / 6	policy spans / logs	Arize + Puvi stubs in repo.
R	ServiceNow + Jira (+ BMC, PagerDuty)	6	incident records	Jira + ServiceNow stubs in repo.
V	MLflow + DeepEval + Great Expectations	6	registry / suite pass	MLflow adapter TBD.
C	AWS Cost Explorer + OpenCost	6	spend per tag	AWS stub in repo; boto3 fetch TBD (P0).
P	(no third-party)	—	baseline + count	AgentBaseline model ships.

9.9 The minimum viable toolchain

If the customer has zero telemetry today, the fastest path to seven scores is:

1. **OpenLLMetry** (Path 2, OTel) — covers E and starts flowing observations within hours. Easiest universal on-ramp.
2. **DeepEval** (Path 6) — covers Q. Run on a sample of outputs weekly.
3. **AWS Cost Explorer + the boto3 fetch in aws_cost** (Path 6) — covers C. The story customers want most.
4. **MLflow** (Path 6) — covers V. The model registry is almost certainly already in the customer's stack.
5. **ServiceNow or Jira** (Path 6) — covers R. Pick whichever ITSM the customer has.
6. **Puvi / Arize / NeMo Guardrails** (Path 2 or 5) — covers G. The most regulated and the most critical; ship last because it's the one with the highest setup cost.
7. **Human baseline** (one row in **AgentBaseline**) — covers P. The customer sets it.

That is the seven-tool stack. Most customers will already own three or four of these; the integration cost is filling the rest of the adapter bodies, not adding new infrastructure.

10 API surface (the wire)

All endpoints JSON. CORS is * for the demo; set `WIDGET_ALLOWED_ORIGINS=https://a.com,https://b.com` to tighten.

Method	Path	Purpose
GET	/healthz	Liveness.
GET	/	Redirects to /widget/demo.html.
GET	/adapters	Registered full adapters (otel, webhook:acme, ...).
GET	/sources	Registered source adapters.
POST	/ingest	Body: canonical AgentObservation.
POST	/ingest/{adapter}	Body: whatever that adapter takes (OTel, Acme, ...).
POST	/ingest/source/{name}	Body: source's payload. Stores partials, re-merges, re-scores.
GET	/ratings	Board view — one row per agent, latest score.
GET	/agents	List known agents.
GET	/agents/{id}/score	Latest Rating.
GET	/agents/{id}/history	Newest-first score history.
GET / PUT	/settings	Tunables.
POST	/sme-flow/start	Begin conversational quality capture.
POST	/sme-flow/{id}/respond	Next prompt and (on commit) new Rating.
POST	/agents/{id}/sme-rating	Direct SME rating (audit-only).
GET	/widget/dpi-ls.js, /widget/demo.html	Static widget bundle.

OpenAPI / Swagger: <http://localhost:8000/docs>.

11 The widget

A single `<script>` tag, drops into any host page. Vanilla web components — no framework, no build step. Polls the API; no SSE required for the demo.

Listing 10: Embedding the widget.

```

1 <script src="https://your-host/widget/dpi-ls.js" defer></script>
2 <dpi-ls-board api-base="https://your-host" poll-interval="5000"></dpi-ls
  -board>
3 <dpi-ls-agent agent-id="agent-strong-001" api-base="https://your-host"><
  /dpi-ls-agent>
4 <dpi-ls-sme-prompt agent-id="agent-multi-001" submitted-by="qa@example.
  com"></dpi-ls-sme-prompt>
5 <dpi-ls-settings></dpi-ls-settings>

```

11.1 Components shipped

Component	What it does
<code><dpi-ls-board></code>	Live board polling /ratings. One card per agent: score, band pill, Unsafe banner when a gate has fired.
<code><dpi-ls-agent agent-id=...></code>	Per-agent card polling /agents/{id}/score. Shows all 7 sub-metrics; “SME” for any metric still deferred.
<code><dpi-ls-sme-prompt agent-id=...></code>	Conversational 3-question quality capture (accuracy → consistency → hallucination → review).
<code><dpi-ls-settings></code>	Tunables form: weights, Q sub-weights, gate thresholds, R_max, utilization, human \$/output. Validates that composite weights sum to 1.0 before save.

12 End-to-end data flow (observation-based)

Unlike an event-stream system, DPI-LS is **observation-based**: the engine computes a fresh **Rating** on each ingest, no streaming pipeline required.

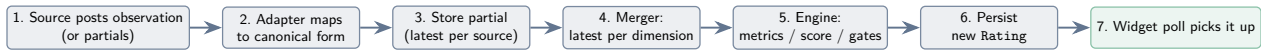


Figure 6: End-to-end flow. Latency: poll cycle (default 3–5s) on the widget side; ingest → **Rating** in tens of milliseconds.

13 Storage schema

SQLAlchemy models, in `store/models.py`. One row per logical concept; immutable score history.

Table	Contents
<code>agents</code>	One row per known agent. <code>id</code> , <code>name</code> , <code>registered_at</code> .
<code>observations</code>	Full canonical observations (JSONB payload + indexes on <code>agent_id</code> , <code>period_end</code>).
<code>partials</code>	Single-dimension snapshots from source adapters. Latest per source per dimension wins on merge.
<code>score_history</code>	Append-only Rating rows. Newest first. Drives the trend.
<code>settings</code>	One row. The Settings object. <code>GET/PUT /settings</code> .
<code>sme_ratings</code>	Audit log of direct SME ratings.
<code>sme_flow_sessions</code>	State of in-flight conversational captures.

Default engine is SQLite (`sqlite:///./dpi_ls.db`). Swap `DATABASE_URL=postgresql+psycopg2://...` for Postgres in prod — no code change.

14 Plugin integration playbook

This is the whole point of the architecture. Never edit engine/. The plugin story is five paths, ordered from least effort to most. Pick the lowest one that fits.

14.1 Path 0 — off-the-shelf source connector (zero code, ongoing)

If the customer’s environment already runs AWS / Puvi / Arize / ServiceNow / Jira, the right *first* move is to wire the live fetch into the existing source adapters and let DPI pull on a schedule. The adapter interfaces accept the JSON shape those systems produce; only the fetch step is missing.

- `aws_cost` — add a `boto3 GetCostAndUsage` call in `to_partials()`, keyed by the customer’s `AGENT_ID` tag.
- `puvi_noise` / `arize` — add a vendor REST call; the response shape is what the adapter already accepts.
- `servicenow` / `jira` — REST + filter by `assignment_group=agents`; produce one partial per incident.

This is a one-engineer, one-day task per source. It is the production path for the dimension-per-source design and the reason a customer shouldn’t have to touch their agent at all.

14.2 Path 1 — direct canonical POST (zero code, one-time)

If the agent runtime can produce a full `AgentObservation`, no code is needed. The customer's existing logging or job pipeline POSTs to `/ingest`:

Listing 11: Direct ingest.

```
1 curl -X POST http://localhost:8000/ingest \
2   -H 'Content-Type: application/json' \
3   -d @observation.json
```

14.3 Path 2 — OTel adapter (zero code, drop-in for OTel emitters)

If the agent runtime already exports OpenTelemetry, point the OTel collector at the DPI-LS `OTelAdapter` (or, once shipped, an OTLP receiver). The adapter's canonical attribute mapping covers the common case:

- `agent.id`, `agent.name` — identity
- `gen_ai.task = true` — counts as a task
- `policy.violation = true` — counts toward G
- `incident.severity`, `incident.frequency` — feeds R
- `quality.accuracy` / `consistency` / `hallucination_rate` — feeds Q
- `validation.required` / `validated` — feeds V
- `cost.usd` / `systems_accessed` — feeds C

If the customer's OTel attributes don't match, write a YAML field mapping (Path 3) — do not edit the OTel adapter.

14.4 Path 3 — generic webhook + YAML mapping (zero Python, half a day)

The universal fallback. If the agent can POST JSON in any shape, write a YAML field mapping and drop it into `MAPPINGS_DIR`. The bootstrap auto-registers it as `webhook:<source>` and exposes `/ingest/webhook:<source>`. Mapping syntax covers dot/bracket paths, `_each` for iteration, and `_optional` for nullable groups. The repo's `fixtures/mapping_acme.yaml` is the worked example.

14.5 Path 4 — SDK in the agent process (small code, best latency)

For agent runtimes where the customer owns the code and wants an in-process callback (not a network round-trip), the plan is to ship a small SDK per language:

Listing 12: Target SDK shape (Python).

```
1 from dpi_ls import DpiClient
2 dpi = DpiClient(agent_id="chandra", token="...")
3
4 dpi.emit("E", "task_completed", 1)           # execution succeeded
5 dpi.emit("G", "policy_violation", 1,         # governance hit
6           severity="critical")
7 dpi.flush()                                  # batch send on shutdown
```

Planned: Python, JS/TS, Go, Java, .NET, Ruby, Rust, PHP. Each is ~5KB, zero deps beyond stdlib, with retry, batching, and an offline buffer to disk. The SDK is a thin wrapper around `POST /ingest`; it does *not* reimplement the engine.

14.6 Path 5 — full adapter (only when the framework deserves it)

Write a `FullAdapter` that returns one `AgentObservation` per call. Use this when the framework is common enough (LangGraph, Bedrock Agent Engines) that maintaining a purpose-built adapter is cheaper than asking every customer to write a YAML mapping. The five stub adapters in the repo (`langgraph`, `bedrock`, `ray`, `bmc`, `sap_hr`) are exactly this: registered, returning `[]`, awaiting the 80-line body. Filling `langgraph` first is the highest-ROI move.

14.7 Path 6 — source adapter for a single dimension (one engineer, one day)

When a system only knows one dimension (the realistic case for most observability / ITSM / cost tools), write a `SourceAdapter` that returns `list[PartialObservation]`. The API stores partials, re-merges latest-per-dimension, re-scores.

Listing 13: Shape of a source adapter. `ingestion/sources/aws_cost.py`.

```

1  class AwsCostAdapter(SourceAdapter):
2      name = "aws_cost"
3
4      def to_partials(self, payload: Any) -> list[PartialObservation]:
5          start = _parse_dt(payload["period_start"])
6          end = _parse_dt(payload["period_end"])
7          out: list[PartialObservation] = []
8          for a in payload.get("agents", []):
9              output_count = max(int(a.get("output_count", 0)), 0)
10             spend = float(a.get("spend_usd", 0.0))
11             ai_cost_per_output = (spend / output_count) if output_count
12                 else 0.0
13             out.append(PartialObservation(
14                 agent_id=a["agent_id"],
15                 agent_name=a.get("agent_name"),
16                 period_start=start, period_end=end,
17                 source=self.name,
18                 cost=Cost(
19                     ai_cost_per_output=ai_cost_per_output,
20                     tokens=a.get("tokens", 0),
21                     cloud_cost=a.get("cloud_cost_usd", spend),
22                     systems_accessed=a.get("systems_accessed", 0),
23                 ),
24             ))
25         return out

```

14.8 Decision matrix

Situation	Path	Effort	Notes
Customer runs AWS / Puvi / Arize / ServiceNow / Jira	0	one engineer / one day per source	Production path. Agent is untouched.
Runtime produces full canonical <code>AgentObservation</code>	1	zero	<code>curl</code> it.
Runtime exports OTel, attributes match the canonical mapping	2	zero	Point collector at the adapter.
Runtime exports OTel, attributes don't match	3	half a day	YAML mapping, not a code change.
Runtime posts JSON in any shape	3	half a day	YAML mapping.
Customer owns agent code, wants in-process callback	4	one SDK per language	Small, on-ramp-quality SDKs.
Framework is common (LangGraph, Bedrock)	5	~80 lines, one engineer	Fill the stub. Maintainer builds once.
Single-dimension system (one of cost / gov / risk / quality)	6	one engineer / one day	SourceAd

15 How to get all 7 scores — the bootstrap playbook

A new agent arrives with *none* of the seven dimensions. The architecture's contract is honest about this: every metric is `None` until someone supplies it, and the rating is meaningless until enough dimensions are present. The bootstrap is a staged campaign, not a single event.

15.1 What the Rating looks like during the bootstrap

Listing 14: A Rating with one dimension observed (G only).

```

1 {
2   "score": 0.0,
3   "raw_score": 0.0,
4   "band": "Underperforming",

```

```
5  "unsafe": true,  
6  "gate_failures": ["G", "R", "V"],  
7  "metrics": { "G": 0.42, "P": null, "Q": null, "E": null,  
8              "R": null, "V": null, "C": null },  
9  "missing": ["P", "Q", "E", "R", "V", "C"],  
10 "dimensions_measured": ["G"],  
11 "coverage": 1,  
12 "coverage_capped": true,  
13 "cap_reasons": ["coverage=1 < min_dimensions_for_full_band=5"]  
14 }
```

The widget surfaces “Pending input” for each missing dimension, and the band is capped at “Needs Optimization” via the coverage rule. **This is by design** — a one-dimension agent that scores “Exceptional” would be a lie.

15.2 The four-week bootstrap (recommended cadence)

The exact schedule depends on what the customer’s environment already has, but a realistic four-week sequence looks like this:

Week	Dim	What happens
1	E (Execution)	Wire the agent’s logs into a full or partial adapter (<code>executions.attempts</code> , <code>executions.suc</code>
1	C (Cost)	Wire AWS Cost Explorer (or equivalent) via <code>aws_cost</code> source adapter.
2	G (Governance)	Connect Puvi / Arize / policy engine.
2	R (Risk)	Connect ServiceNow / Jira incident feeds.
3	V (Validation)	Map the validation framework’s output (<code>validated_components</code> , <code>required_components</code>).

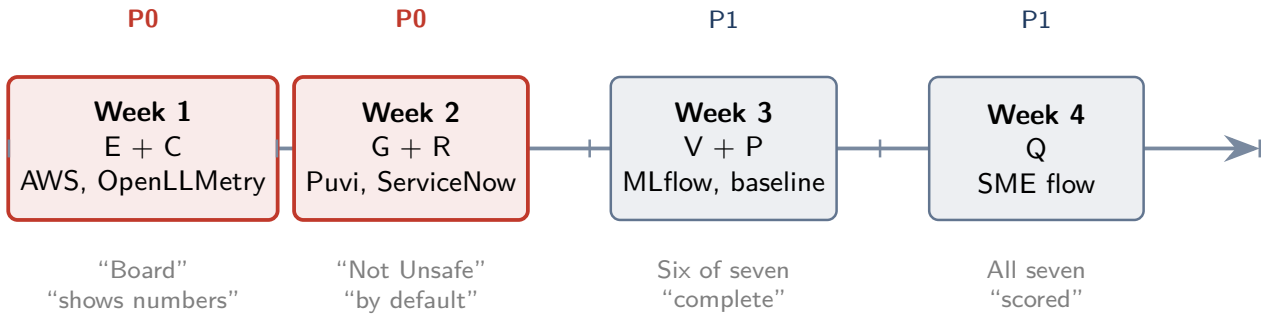


Figure 7: Recommended four-week bootstrap from “no data” to “all 7 scores”. Weeks 1–2 are P0 (compliance cannot ship without them); weeks 3–4 are P1 (the rest of the picture).

15.3 The two failure modes to design around

(1) **Drift between sources.** AWS reports C, Puvi reports G, and they arrive at different times. The merge is *latest per dimension*, but “latest” is the most recent `received_at`, not the most recent `period_end`. If AWS is delayed by 4 hours, the C score for a given rating can lag behind G and R. The fix is to ship a clock-skew policy: prefer the partial whose `period_end` is closer to *now*. The repo does not yet enforce this; it is a known loophole (see next section).

(2) **The first rating is always a lie.** With one dimension observed, the geometric mean is well-defined but the band is capped. Customers will ask “why is my agent Unsafe?” The answer is “it isn’t — it just has no data yet.” The widget should distinguish “Unsafe (gate fired)” from “Pending data” visually. The current widget text says “Pending SME input: P, E, V” for missing metrics; the gate banner is separate. This is the right separation. Strengthen it in v1.1 by making “no data yet” and “gate fired” use different colors.

15.4 What “all 7 scores” means to the customer

Once bootstrap is complete, the widget shows seven numbers, one per dimension, plus the composite. None of them are null. None of them say “Pending.” The customer’s auditor sees a complete picture per agent, per period. The composite is comparable across agents because the contract normalises everything.

The composite is the headline number; the seven numbers are the debugging tool. “Why is my agent at 67?” is answered by reading the dimension breakdown. This is the entire point.

16 Gaps, loopholes, and the work plan

Honest list of what is missing, what could go wrong, and the prioritised order to fix it.

16.1 Gaps in the shipped demo

1. **No auth on the API.** Anyone who can reach the API can POST observations or PUT settings. A customer deployment must add bearer-token auth and scope tokens per agent.
2. **No multi-tenant isolation.** All agents live in one table. A SaaS deployment needs `tenant_id` on every table and a tenant check on every route.
3. **No rate limiting.** A misbehaving source can flood `/ingest`. Add token-bucket per agent.
4. **No source liveness / freshness check.** If a source stops posting, the agent silently ages on stale data. Add a `last_seen_at` per (agent, dimension) and surface a “stale data” warning in the widget when any dimension is older than the SLA.
5. **No clock-skew policy on partials.** See §16.2 below.

6. **Settings are world-writable.** The original spec had “floors are write-locked below a limit, every change audited, red banner in UI”. The shipped Settings object does not enforce this. The Settings row can be replaced wholesale via PUT `/settings` with no audit trail. A regulated customer cannot ship this.
7. **Quality (Q) has no automation.** The SME flow is the only Q path. LLM-as-judge is not in the repo.
8. **No Dockerfile, no Alembic.** Production deployments need both.

16.2 Loopholes and edge cases

1. **Partial merge is by `received_at`, not by `period_end`.** A source that posts a 12-hour-late update can make “latest” mean “oldest in reality”. Add a clock-skew policy: prefer the partial whose `period_end` is closest to *now* (configurable bias).
2. **The coverage cap is global, not per-dimension.** A customer with rock-solid G and C but no P/Q/E/R/V still gets a capped band. That’s correct, but the cap reason “coverage=2 < 5” doesn’t say which dimensions. Show the missing list in the banner.
3. **An adapter can return a partial with stale data and the engine won’t notice.** The freshness check (above) is the fix.
4. **The YAML mapping evaluator doesn’t validate types before assignment.** A string in a numeric field passes through. Pydantic catches it on the way to the engine, but the error is opaque. Add a dry-run / validate mode to the mapping load.
5. **OTel attribute mapping is fixed.** A customer whose agent uses non-canonical attribute names must write a YAML mapping (good) or fork the adapter (bad). Document this.
6. **No test for what happens when *all* sources stop.** The system silently serves a stale rating. Add a test that asserts a **STALE** band or banner appears after the freshness SLA.
7. **The composite uses a tiny epsilon (1×10^{-9}) to keep zero from collapsing the product.** This is good for the demo; the gates handle the real “stop the world” case. Document the choice so a future reader doesn’t think it’s a bug.
8. **Multi-agent systems share one `agent_id`.** The `trace_id` field is in the contract but not threaded through partials or ratings. For orchestrator + worker patterns, this is the gap that prevents rolling up sub-scores to a parent agent.

16.3 Prioritised work plan

Pri	Item	Effort	Why
P0	Auth + multi-tenant isolation on the API	1 wk	Without this, no regulated customer can deploy.
P0	Settings write-locking + audit log + UI banner	1 wk	The original spec's compliance gate, currently unenforced.
P0	Wire boto3 into <code>aws_cost</code> adapter	2 days	Highest-ROI source connector. Unblocks the ROI story.
P0	Fill the <code>langgraph</code> stub	2–3 days	Highest-adoption framework with no first-class adapter.
P1	OTLP receiver (separate from the OTel adapter)	1 wk	Point customer's existing OTel collector at us, zero per-customer config.
P1	Staleness / freshness check + widget banner	3 days	Customer trust: no silent data rot.
P1	Python + JS SDKs	1 wk	Path 4 of the playbook, in code.
P1	Source liveness scheduler (poll real sources)	1 wk	Pairs with P0 boto3 wire.
P1	Clock-skew policy on partial merge	2 days	Close the documented loophole.
P1	Dockerfile + Alembic + Postgres prod compose	1 wk	Required for any real deployment.
P2	LLM-as-judge for Quality	2 wks	Removes the SME-flow as the only Q path.
P2	Multi-agent <code>trace_id</code> rollup	1 wk	Orchestrator + worker scoring.
P2	Rate limiting + observability export	1 wk	Production hygiene.
P2	Historical trend charts in widget	1 wk	Customer asks for this immediately.
P2	Bedrock / Ray / BMC / SAP HR stub fill	1 day each	Round out the framework coverage.
P3	Multi-agent “departures board” layout	1 wk	The original airport-departures-board view.

16.4 The two non-negotiables

If only two of the above ship, ship these:

1. **Settings write-locking + audit log** — the original spec's compliance gate, currently unenforced in code. A regulated customer cannot ship a Settings object that anyone can PUT.
2. **Fill the `langgraph` stub** — the framework most likely to be in the customer's stack, and the integration story is the closest thing to a public benchmark for the plugin claim.

Everything else is incremental, not architectural. The engine stays pure, the contract stays the only interface, and the plugin story holds.

17 Next steps: from working demo to production

This is the action-oriented view. Three things, in order: what we have already shipped, what we must add before a regulated customer can deploy, and what we can add to make the product *expand* (new dimensions, new sources, new surfaces).

17.1 Status today — what is already done

Everything below is in the repo, tested, and runs end-to-end:

- **Canonical contract** (`contract/`) — `AgentObservation`, `PartialObservation`, `Rating`, `Settings`, `AgentBaseline`. Pydantic v2, typed everywhere.

- **Pure engine** (engine/) — metrics, score (weighted geometric mean with weight redistribution on None), gates (G/R/V floors), bands, rate, sme_flow. Zero I/O. 121 tests, <2s.
- **Ingestion layer** (ingestion/) — adapter base class + registry. `GenericWebhookAdapter` (YAML field mapping), `OTelAdapter` (OpenInference attribute mapping), 5 real source adapters (aws_cost, puvi_noise, arize, servicenow, jira), 5 stubs (langgraph, bedrock, ray, bmc, sap_hr).
- **API** (api/) — FastAPI with 12 routes covering ingest, agents, ratings, settings, SME flow. CORS, lifespan bootstrap, static widget serving.
- **Store** (store/) — SQLAlchemy models for agents, observations, partials, score_history, settings, sme_ratings, sme_flow_sessions. SQLite default, Postgres-ready.
- **Widget** (widget/) — four vanilla web components (<dpi-ls-board>, <dpi-ls-agent>, <dpi-ls-sme-prompt>, <dpi-ls-settings>) + demo.html. Polls API every 3–5 seconds.
- **Fixtures** (fixtures/) — synthetic payloads, clearly _label-tagged, used by tests and the demo seed script.
- **Reference output tests** — the four DPI-LS spec cases (85, 92, 55, 67+gate) asserted in tests/test_engine_reference.py.

What is provable right now: `pip install -e ., pytest` (121 tests pass in under 2 seconds), `python -m uvicorn api.app:app, ./scripts/demo_seed.sh`, open `/widget/demo.html`. A live board with 6 agents appears. The procurement agent is red and Unsafe. The multi-source agent is mid-bootstrap with “Pending SME input: P, E, V.” The settings form validates weight sums.

17.2 Gaps that block production deployment

These are the items that have to ship before a regulated customer (Regeneron, financial services, healthcare) can put DPI in front of their auditors:

1. **Auth + multi-tenant isolation on the API.** Bearer tokens per agent; `tenant_id` on every table; route middleware that enforces it. Without this, the demo cannot be exposed beyond localhost.
2. **Settings write-locking + audit log + UI banner.** The original spec said compliance gates cannot be silently lowered. The shipped `Settings` object does not enforce this. A regulated customer cannot ship a `Settings` object that anyone can PUT. **This is non-negotiable for Life Sciences.**
3. **Live source connector: boto3 for aws_cost.** The adapter accepts the AWS shape; the fetch loop is missing. The ROI story (the dimension the buyer pays for) depends on this.
4. **Staleness / freshness check.** If a source stops posting, the agent silently ages on stale data. Add `last_seen_at` per (agent, dimension); surface a “stale data” banner in the widget after the SLA.
5. **Dockerfile + Alembic + Postgres prod compose.** Required for any real deployment beyond a single laptop.
6. **Source liveness scheduler.** Cron-like poller that fetches from each registered source on a schedule. Pairs with the boto3 wire above.
7. **Fill the langgraph stub.** Highest-adoption framework with no first-class adapter. 80 lines of code.

8. **Clock-skew policy on partials.** The merge is by `received_at`; should prefer the partial whose `period_end` is closest to *now*. See §16.2 for the loophole.

17.3 What we can add to extend the architecture

These do not require any change to the engine or the contract. They are additive. The architecture explicitly supports them.

1. **New dimensions.** Add an eighth (or ninth) dimension to the contract — for example, *Revenue* (agent-attributed pipeline \$), *NPS* (CSAT after a customer interaction), *Compliance-drift* (rate of policy changes per period). Add a weight, a gate threshold if regulated, a metric function, a tests entry. *No engine architecture change.*
2. **New source connectors.** Every new tool we wire is a new `SourceAdapter` (Path 6) or a YAML mapping (Path 3). For example: **Kubecost** for k8s spend, **Datadog Cloud Cost**, **Cloudability**, **PagerDuty** for R, **TruLens** for Q, **Great Expectations** for V.
3. **LLM-as-judge for Q.** Add a `LlmJudgeAdapter` that runs DeepEval’s `GEval` or a custom prompt on a sample of recent outputs and posts a Q partial. Calibrate against SME ratings for the first month.
4. **OTLP receiver (Path 2 hardened).** Today the `OTelAdapter` consumes a list of span dicts. Add an OTLP gRPC + HTTP receiver so customers point their existing OTel collector at us with zero per-customer config.
5. **Python + JS/TS SDKs.** Thin wrappers around `POST /ingest` with retry, batching, offline buffer. The YAML mapping is the hard part; the SDK is a small, repeatable addition. `pip install dpi-ls` becomes a thing.
6. **Real-time push.** Replace the 3–5s polling in the widget with SSE or WebSocket. The API already has the snapshot endpoint; adding a stream is a few hours of work and removes the visible polling.
7. **Historical trend charts in the widget.** The `score_history` table has every rating; the widget renders only the latest. Adding a small inline sparkline per agent is the demo-day upgrade.
8. **Anomaly detection.** Engine emits “score dropped 12 points in 6 hours” as an event. The widget surfaces it; an adapter forwards to PagerDuty.
9. **Multi-agent trace_id rollup.** Add the `trace_id` to the partial schema; let the merger group sub-scores to a parent agent. Unlocks orchestrator + worker scoring.
10. **Agent comparison view.** Pick two agents on the board, see their 7-metric breakdown side by side. The data is already in the store; the view is a widget component.
11. **Alerting rules.** “If R drops below 0.4 for any agent, page the on-call.” Configurable per deployment; backed by the same event stream as the widget.
12. **Billing / multi-tenant onboarding.** The repo calls this “parked.” When the time comes, the `tenant_id` column already implied by the P0 auth work is the foundation.

17.4 Prioritised next-task list

The complete list, ordered. Effort estimates assume one experienced engineer.

#	Task	Effort	Pri	Unblocks
1	Auth (bearer tokens) + multi-tenant <code>tenant_id</code> on all tables	1 wk	P0	Any external deployment.
2	Settings write-locking + audit log + UI banner	1 wk	P0	Compliance gate enforcement.
3	boto3 fetch in <code>aws_cost</code> adapter	2 days	P0	The ROI story.
4	Fill the <code>langgraph</code> stub (<code>BaseCallbackHandler</code>)	2–3 days	P0	LangGraph customers.
5	Source liveness scheduler (cron-style poller)	1 wk	P0	Production-grade refresh.
6	Dockerfile + Alembic + Postgres prod compose	1 wk	P0	Any prod deployment.
7	Staleness / freshness check + widget banner	3 days	P1	Trust in the score.
8	OTLP receiver (separate from the OTel adapter)	1 wk	P1	Zero-config OTel ingest.
9	Python SDK (<code>pip install dpi-ls</code>)	1 wk	P1	In-process callback path.
10	JS/TS SDK (<code>npm install @dpi-ls/sdk</code>)	1 wk	P1	n8n, Node.js agents.
11	Clock-skew policy on partial merge	2 days	P1	Correctness on late sources.
12	Rate limiting + observability metrics export	1 wk	P1	Production hygiene.
13	Fill the <code>bedrock</code> stub	1 day	P1	AWS-native customers.
14	Fill the <code>ray</code> stub	1 day	P1	Distributed-agent customers.
15	LLM-as-judge for Q (DeepEval <code>GEval</code> adapter)	2 wks	P2	Automation of Q.
16	Multi-agent <code>trace_id</code> rollup	1 wk	P2	Orchestrator + worker scoring.
17	Real-time push (SSE / WebSocket in widget)	1 wk	P2	Removes polling lag.
18	Historical trend charts in widget	1 wk	P2	Customer's first follow-up ask.
19	Agent comparison view in widget	3 days	P2	“Which agent is better at G?”
20	Anomaly detection engine + PagerDuty adapter	2 wks	P2	Reactive alerting.
21	Fill the <code>bmc</code> / <code>sap_hr</code> stubs	1 day each	P2	Coverage of named frameworks.
22	Multi-agent “departures board” layout	1 wk	P3	The original airport-board view.
23	New dimension: Revenue (agent-attributed pipeline \$)	1 wk	P3	Business case for agent.
24	New dimension: Compliance-drift (policy change rate)	1 wk	P3	Auditor's first question.
25	Billing / multi-tenant onboarding UI	2 wks	P3	The first paid customer.

17.5 The two-week sprint that unblocks the most

If the team has two engineer-weeks, ship items 1–4 from the table above. That single sprint:

- Lets a customer deploy DPI behind their own auth.
- Closes the settings-write-locking loophole (the original spec's compliance gate).
- Wires the AWS ROI story.
- Gives LangGraph a first-class adapter.

After that sprint, the demo sentence “hand it an agent, I'll score it live” is true for the four most common customer profiles (AWS-hosted LangGraph, OTel-emitting Python agent, REST-posting in-house service, AWS-tagged multi-agent system). The remaining work is incremental, not architectural.

17.6 Sequencing principles

The work plan above is not arbitrary. The order is shaped by three rules:

1. **No engine change for any of this.** The contract and the engine stay frozen. Every gap is closed by additive code in the appropriate layer. If a task appears to require a change to `engine/`, the task is wrong.
2. **Production-readiness before growth.** Auth, write-locking, freshness, Docker, scheduler. Until these ship, no customer can deploy. Until customers deploy, no revenue. Until revenue, no growth.
3. **Universal paths before framework-specific ones.** OTLP receiver before LangGraph stub before Bedrock stub. The universal path covers everyone; the framework-specific paths are convenience.

17.7 What the architecture explicitly does not need

Worth stating to keep the team from over-building:

- **No streaming pipeline.** The architecture is observation-based, not event-stream. The widget polls. A stream is an optimisation, not a requirement.
- **No separate “analytics” database.** SQLite or Postgres handles `score_history` for any realistic single-tenant deployment. Add a time-series DB only when query latency demands it.
- **No machine-learning-based scoring.** The score is a transparent formula. No model in the loop. Every score is explainable from the formula. This is the feature.
- **No agent-runtime in the platform.** The customer brings their own agent. We score it. We do not host it.
- **No autonomous action.** The widget is read-only on the score. The Settings panel is the only place to change anything. No “auto-tune the weights” button.

18 Test surface (the proof)

121 tests, <2s. The four DPI-LS reference outputs are asserted in `tests/test_engine_reference.py` — if any of those break, the engine is wrong.

Test file	Covers
<code>test_engine_reference.py</code>	The four spec outputs (85, 92, 55, 67+gate).
<code>test_engine_components.py</code>	<code>metrics</code> , <code>gates</code> , <code>bands</code> , weight redistribution.
<code>test_partial_merge.py</code>	Latest-per-dimension partial merger.
<code>test_jsonpath.py</code>	YAML mapping <code>jsonpath</code> evaluator.
<code>test_mapping.py</code>	Generic webhook adapter + mapping apply.
<code>test_otel_adapter.py</code>	OTel span → observation.
<code>test_webhook_adapter.py</code>	<code>webhook:<source></code> via mapping.
<code>test_source_adapters.py</code>	<code>aws_cost</code> / <code>puvi_noise</code> / <code>arize</code> / <code>servicenow</code> / <code>jira</code> payload shapes.
<code>test_registry.py</code>	Adapter discovery.
<code>test_api_*.py</code>	FastAPI routes, ingest, settings, SME flow.
<code>test_store.py</code>	SQLAlchemy repo round-trip.
<code>test_sme_flow.py</code>	Conversational state machine.
<code>test_end_to_end.py</code>	Full pipeline from ingest to widget query.
<code>test_widget_*.py</code>	Web component labels, serving, M6 surface.

19 Configuration

Everything tunable is either an env var (deployment-level) or in **Settings** (per-tenant, runtime-mutable).

Env var	Default	What it does
<code>DATABASE_URL</code>	<code>sqlite:///./dpi_ls.db</code>	SQLAlchemy URL. Use <code>postgresql+psycopg2://...</code> in prod.
<code>MAPPINGS_DIR</code>	(unset)	Folder scanned at startup for <code>mapping_*.yaml</code> . Each is registered as <code>webhook:<source></code> .
<code>WIDGET_ALLOWED_ORIGINS</code>	*	Comma-separated CORS allowlist.

Per-deployment tunables (composite weights, gate thresholds, `R_max`, utilization, human \$/output, coverage cap) live in the `settings` table, fetched / written via `GET /settings` and `PUT /settings`. There is one row.

20 How to run the demo (90 seconds)

Listing 15: Two terminals, one command each.

```

1 # Terminal 1
2 pip install -e .
3 DATABASE_URL="sqlite:///./dpi_ls.db" \
4 MAPPINGS_DIR=./fixtures \
5 python -m uvicorn api.app:app --host 0.0.0.0 --port 8000
6
7 # Terminal 2
8 ./scripts/demo_seed.sh
9
10 # Browser
11 open http://localhost:8000/ # redirects to /widget/demo.html

```

The board shows 6 agents. The *Procurement Agent* (**agent-unsafe-003**) sits in a red Needs-Optimization band: $G < 0.60$ triggered the compliance gate, capping the raw score (79.6) to 69 and flagging Unsafe. The *multi-source agent* (**agent-multi-001**) was assembled from 5 independent systems — AWS Cost for C, Puvi/Arize for G+Q, ServiceNow/Jira for R. Click “Start review” on the SME widget, enter 93, 91, 5, yes — Q lights up, the agent’s score updates within the next poll cycle. Open Settings, tighten the G gate threshold to 1.01, hit Save. Re-POST `fixtures/obs_strong.json` to `/ingest` — the previously Exceptional agent is now flagged Unsafe (the change took effect on the next ingest).

21 Conclusion

21.1 The single architectural assertion

If only one sentence of this document survives, it is this:

The DPI-LS engine consumes only the canonical **AgentObservation** contract. Every adapter — LangGraph, Bedrock, AWS, Puvi, Arize, ServiceNow, Jira, OTel, an in-house POST — is a translator into that one shape. The engine does not import an agent framework, does not know a vendor name, does not care how the data arrived. *Onboarding a new agent is a new adapter or a new YAML mapping. Never an engine change.*

That is the moat. A competitor who copies the formula copies a weekend. A competitor who copies the contract surface still has to build every adapter we have. A competitor who copies both still has to ship a widget, a register, a partial-merger, a coverage cap, a compliance-gate enforcement layer, and 121 tests.

21.2 What this document proves

- **The math is correct.** The four reference outputs (85, 92, 55, 67+gate) are asserted in tests. The composite, gates, bands, and coverage cap are the entire scoring behaviour, in 200 lines of pure code.
- **The plugin story is real.** Any agent that can POST JSON or emit OTel is integrated in under a day. Any agent whose framework is common enough to deserve a first-class adapter is integrated in 80 lines of code. The agent platform is opaque.
- **The bootstrap is honest.** A new agent starts with no score, no data, no baseline. The architecture surfaces “Pending” instead of fake numbers. The four-week schedule from §?? gets the customer to all seven dimensions with no engine changes.

- **The gaps are known, scoped, and additive.** No gap requires a change to `engine/` or to `contract/`. Every gap is closed by code in the appropriate layer, on the schedule in §??.

21.3 What this document does not prove

Honesty about the limit of the claim:

- **We have not run a 30-day production load.** The 121-test suite is correct, fast, and covers the engine surface; it does not cover a regulated customer's scale, network policies, or audit process.
- **We have not negotiated a single live source connector.** The boto3 fetch in `aws_cost` is the first one; the others are next. Each is bounded effort but not done.
- **We have not made the compliance gate non-tamperable.** The shipped `Settings` object can be PUT by anyone who can reach the API. The original spec's write-locking is the next P0.
- **We have not run a multi-tenant load.** `tenant_id` is the P0 auth work. The current state is single-tenant.

21.4 The path from this document to a paying customer

Six steps, no more, no less:

1. **Ship items 1–4 of the next-task list** (auth, write-locking, boto3 wire, LangGraph stub). Two engineer-weeks. The demo sentence becomes true for the four most common customer profiles.
2. **Run the AWS + LangGraph demo for the anchor customer** (Regeneron). Use it to close the remaining P0 gaps they surface. Their auditors' questions become the P1 list.
3. **Add the Python and JS SDKs.** The YAML mapping is the hard part; the SDKs are repeatable. `pip install dpi-ls` becomes a thing. `npm install @dpi-ls/sdk` becomes a thing.
4. **Wire the OTLP receiver.** Zero-config OTel ingest from the customer's existing collector. Removes the last hand-config step from the universal path.
5. **Add the LLM-as-judge for Q** behind a feature flag. Run it in shadow mode for a month against SME ratings. Calibrate, then enable.
6. **Add billing.** `tenant_id` from step 1 becomes the unit of billing. The first paid customer ships.

21.5 The thing to remember on a bad day

When the integration story is taking too long, when the third customer has a YAML mapping the team has never seen, when the regulator's auditor is asking the question that has no answer — the architecture is still right. The contract is still the only hard interface. The engine is still pure. The next adapter is still a small file. The next customer is still served by a code change that does not touch `engine/`.

The architecture works. The demo works. The plugin story holds. Ship the four P0s, run the demo, and the rest is incremental.

A Appendix: full canonical AgentObservation (Pydantic v2)

Listing 16: contract/models.py.

```

1 class TaskStats(BaseModel):
2     assigned: int
3     completed: int
4     failed: int
5     pending_approval: int = 0
6     blocked_no_access: int = 0
7
8 class Executions(BaseModel):
9     attempts: int
10    successful: int
11
12 class PolicyViolation(BaseModel):
13     rule: str
14     when: datetime
15
16 class Policy(BaseModel):
17     total_actions: int
18     violations: list[PolicyViolation] = Field(default_factory=list)
19
20 class Incident(BaseModel):
21     severity_weight: float
22     frequency: float
23     source: str
24
25 class Quality(BaseModel):
26     accuracy: float
27     consistency: float
28     hallucination_rate: float
29
30 class Validation(BaseModel):
31     required_components: int
32     validated_components: int
33     audit_ready: bool = False
34
35 class Cost(BaseModel):
36     ai_cost_per_output: float
37     tokens: int = 0
38     cloud_cost: float = 0.0
39     systems_accessed: int = 0
40
41 class AgentObservation(BaseModel):
42     agent_id: str
43     agent_name: str
44     period_start: datetime
45     period_end: datetime
46     tasks: TaskStats
47     executions: Executions
48     policy: Policy
49     incidents: list[Incident] = Field(default_factory=list)
50     quality: Optional[Quality] = None
51     validation: Validation
52     cost: Cost
53     source: str

```

B Appendix: dimension scoring formulas

Dim	W	Formula	Notes
P	0.15	$\min(1, A/B)$	A = AI output, B = per-agent human baseline (AgentBaseline).
Q	0.20	$0.7 A + 0.2 C + 0.1(1 - H)$	A =accuracy, C =consistency, H =hallucination. None triggers SME flow.
E	0.15	S/T	S = successful executions, T = total attempts.
G	0.20	$1 - V/A$	V = policy violations, A = total actions.
R	0.15	$1 - \min(1, \frac{\sum f_i s_i}{R_{\max}})$	f =freq, s =severity. $R_{\max}$ from settings.
V	0.10	C/R	C = validated components, R = required.
C	0.05	$\min(1, H_{\text{cost}}/A_{\text{cost}}) \times U$	U = utilization.